



Department of Electronic & Telecommunication Engineering,
University of Moratuwa, Sri Lanka.

Assignment 2

Application of 2D-DCT for Image Compression

230155J - Dissanayake D. M. D. P.
EN3551 - Digital Signal Processing

Date: October 9, 2026

3.1 Comparison of Original and Reconstructed Images

Barbara Image

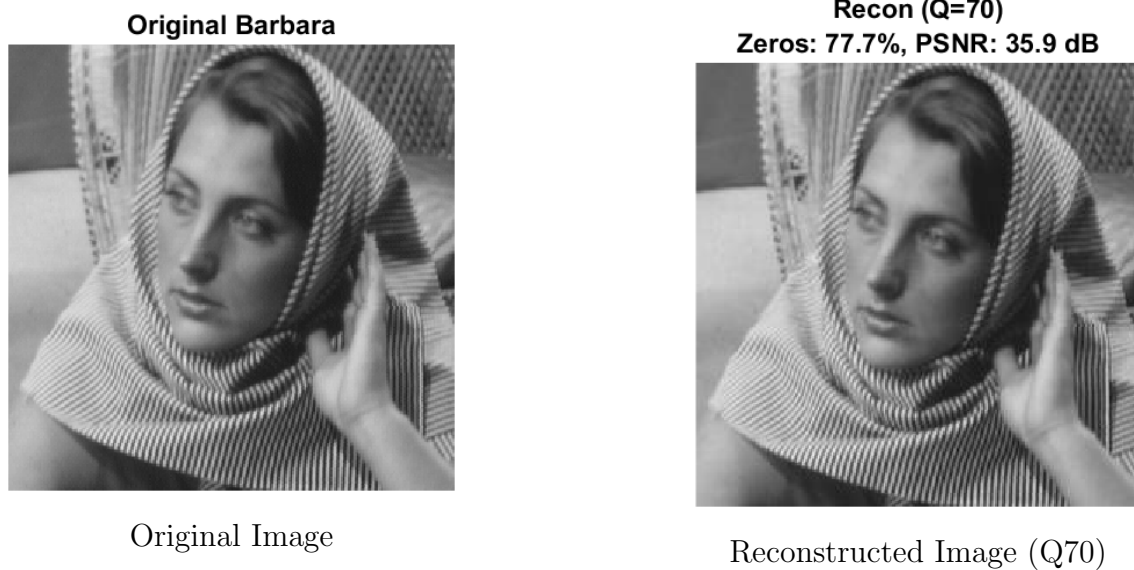


Figure 1: Comparison of Original and Reconstructed Barbara Image

Discussion: The reconstructed Barbara image at Q70 retains most of the original details and there is minimal artifacts in sharp details. The edges and textures are well preserved effective compression without significant loss of quality. PSNR is 35.9dB indicating a better reconstruction.

Boats Image

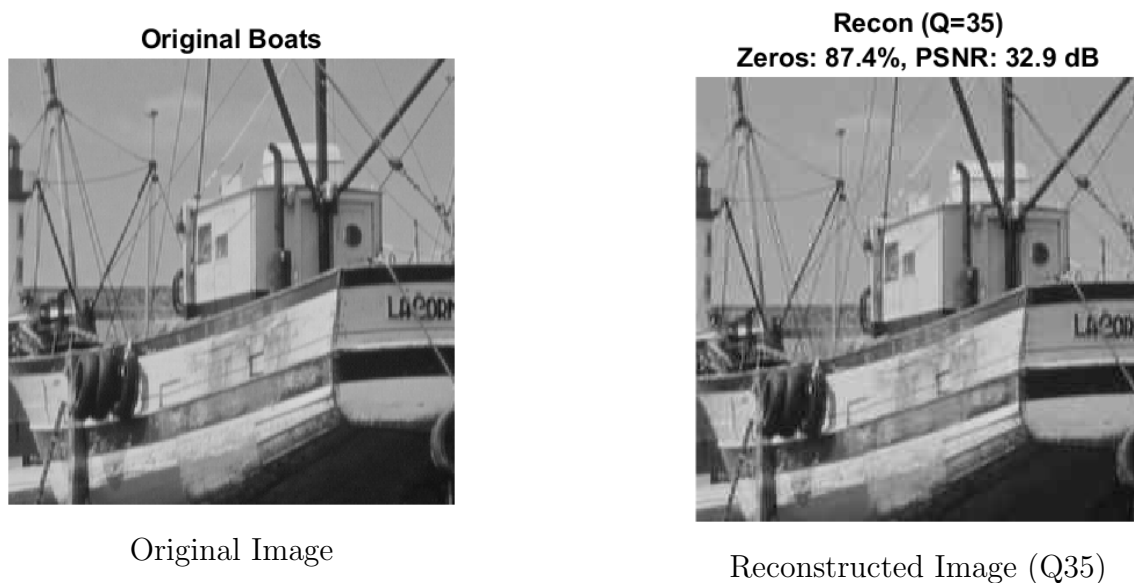


Figure 2: Comparison of Original and Reconstructed Boats Image

Discussion: The reconstructed Boats image at Q35 shows a noticeable loss of detail compared to the original. While the overall structure is maintained. But the finer textures and edges are blurred. This can be explained by the percentage of zeroes per 8x8 block which is about 87.4%. The PSNR of the reconstructed image is 32.9dB and this confirms the slight difference that can be seen visually.

Peppers Image

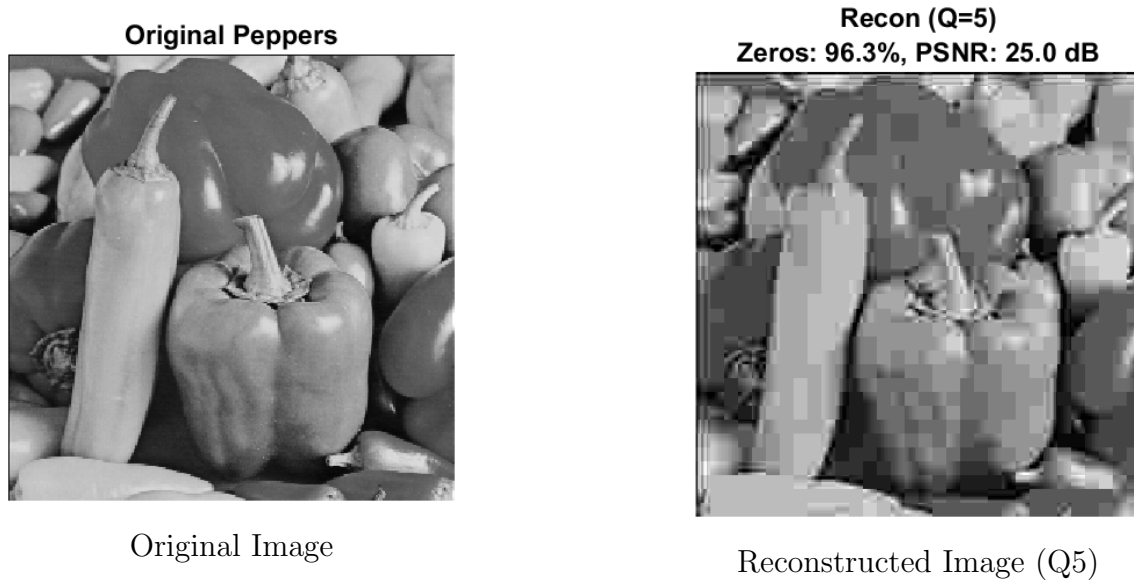


Figure 3: Comparison of Original and Reconstructed Peppers Image

Discussion: The reconstructed Peppers image at Q5 shows significant loss of detail and quality. The image appears blurry and square like blocks are clearly visible. At this low quality level it results in a high percentage of zeroes per 8x8 block (about 96.3%). The compression is effective in reducing file size but the visual quality is compromised. The PSNR value is below 30dB so the structures of the reconstructed image is not preserved well and it can be seen by the human eye.

Sigiriya Image (Additional Image)

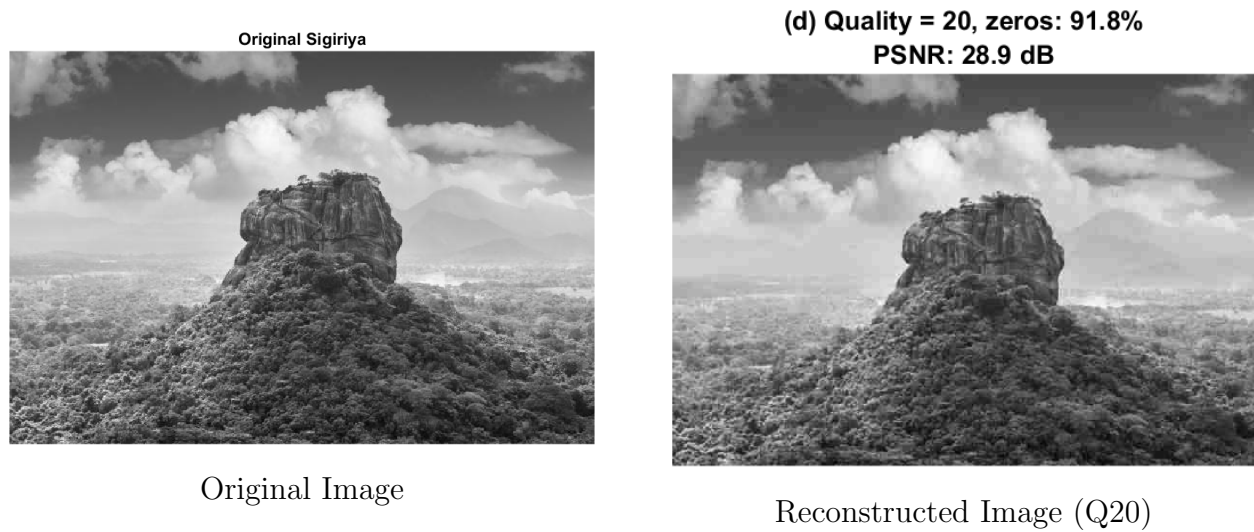


Figure 4: Comparison of Original and Reconstructed Sigiriya Image

Discussion: The reconstructed image at 20% quality level shows very little visual differences compared to the original image. Percentage of zeroes is 91.8%. Sharp and high frequency content like trees and corners are somewhat affected, but the overall image is not much affected. This may be due to the higher height and width of image (608 * 408 px)

Sweeping Different Quality Levels for Sigiriya Image

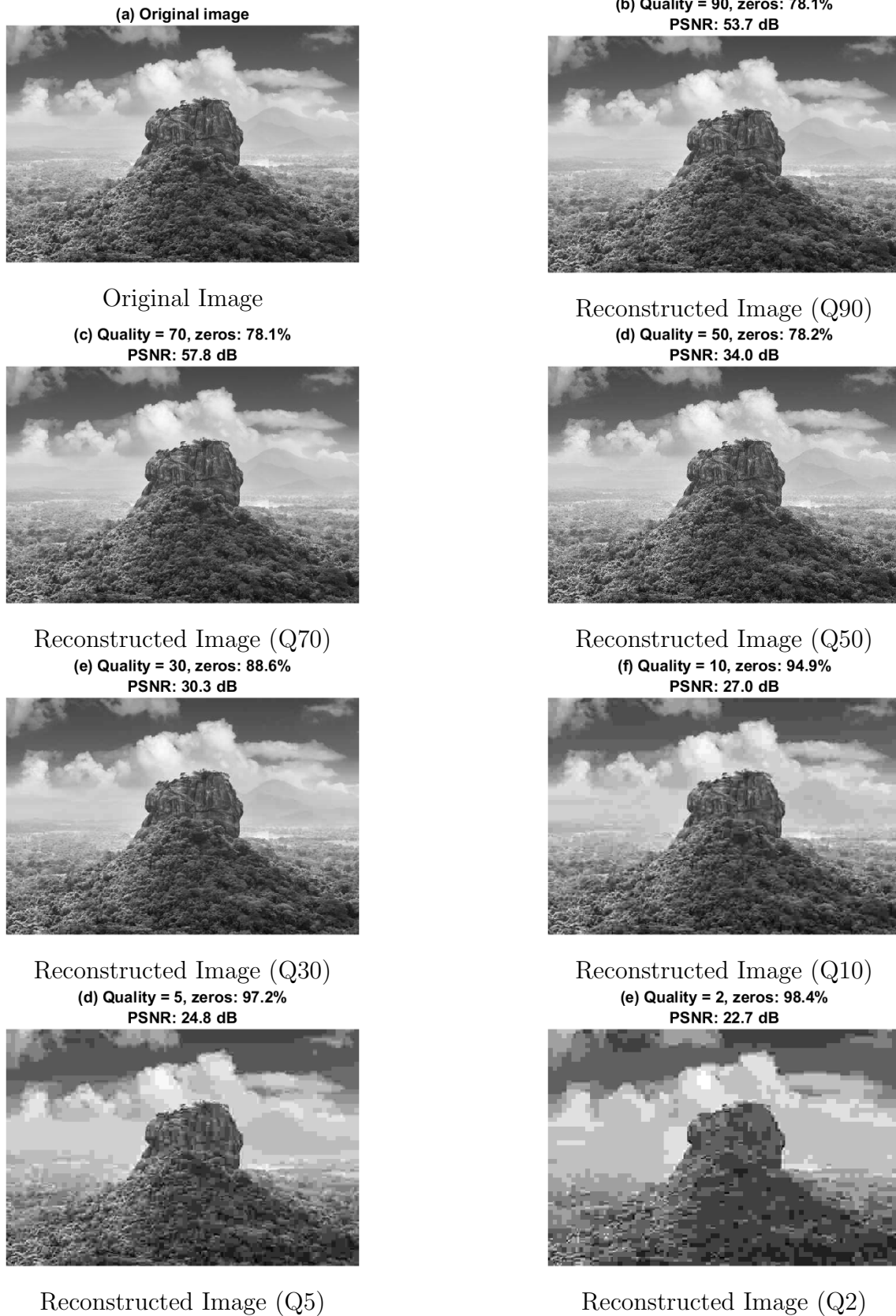


Figure 5: Comparison of Reconstructed Sigiriya Images at Different Quality Levels

3.4 Discussion on the Effect of Quality Levels on different Images and compression

The quality level in image compression using 2D-DCT significantly affects the reconstructed visual quality. Higher quality levels retain more of the original image's details and textures. It results in a reconstructed image that closely resembles the original. Conversely, lower quality levels (e.g., Q5) lead to more aggressive quantization, which can introduce artifacts and loss of fine details, making the reconstructed image appear blurry or pixelated.

- Images with high frequency content are difficult to compress without losing quality due to the sharp transitions like edges and textures. After compression these fine details can be blurred or pixelated.
- Plain images with not significant transitions like edges and textures can be compressed without pixelation. So these images can be considered as easier to compress. As an example, the Sigiriya Image has **sky and clouds** which are well preserved at lower qualities but the sharp details in trees and textures in rock are visibly pixelated at very lower quality levels.
- The percentage of zeroes per 8x8 block is a good indicator of the compression level. Higher percentages indicate more aggressive compression, which can lead to loss of detail and visual similarity. The Peppers image at Q5 has a high percentage of zeroes (96.3%) which results in significant loss of quality at least on plain surfaces.

Appendix A: MATLAB Implementation

Listing 1: Sample MATLAB script for 2D-DCT image compression

```

1  %% Load the data
2  barbara_data = load('Images/barbara.mat');
3  boats_data   = load('Images/boats.mat');
4  peppers_data = load('Images/peppers.mat');
5  sigiriya_data = load('Images/sigiriya.mat');
6
7  % Get the actual variable names
8  f_barb      = fieldnames(barbara_data);
9  f_boat      = fieldnames(boats_data);
10 f_pepp      = fieldnames(peppers_data);
11 f_sigiriya  = fieldnames(sigiriya_data);
12
13 % Extract the images
14 barb_img     = barbara_data.(f_barb{1});
15 boat_img     = boats_data.(f_boat{1});
16 pepp_img     = peppers_data.(f_pepp{1});
17 sigiriya_img = sigiriya_data.(f_sigiriya{1});
18
19
20 % Convert colour to grayscale
21 if size(sigiriya_img, 3) == 3
22     sigiriya_img = rgb2gray(uint8(sigiriya_img));
23 end
24
25
26 % Standard Q50 matrix (GIVEN)
27 Q50 = [16 11 10 16 24 40 51 61;
28        12 12 14 19 26 58 60 55;
29        14 13 16 24 40 57 69 56;
30        14 17 22 29 51 87 80 62;
31        18 22 37 56 68 109 103 77;
32        24 35 55 64 81 104 113 92;
33        49 64 78 87 103 121 120 101;
34        72 92 95 98 112 100 103 99];
35
36 Q_barbara    = getQuantMatrix(Q50, 70);
37 Q_boats      = getQuantMatrix(Q50, 35);
38 Q_peppers    = getQuantMatrix(Q50, 5);
39 Q_sigiriya   = getQuantMatrix(Q50, 30);
40
41 %% Evaluate the three test images + sigiriya
42 [recon_b, zeros_b, psnr_b] = evaluate_compression(barb_img,
43     Q_barbara);
44 [recon_o, zeros_o, psnr_o] = evaluate_compression(boat_img,
45     Q_boats);

```

```

44 [recon_p, zeros_p, psnr_p] = evaluate_compression(pepp_img,
    Q_peppers);
45 [recon_s, zeros_s, psnr_s] = evaluate_compression(sigiriya_img,
    Q_sigiriya);
46
47 % Compare original vs reconstruction for the three test images
48 figure;
49 subplot(2,3,1); imshow(uint8(barb_img)); title('Original Barbara');
50 subplot(2,3,2); imshow(uint8(boat_img)); title('Original Boats');
51 subplot(2,3,3); imshow(uint8(pepp_img)); title('Original Peppers');
52 subplot(2,3,4); imshow(uint8(recon_b)); title(sprintf('Recon (Q=70)\n
    nPSNR: %.1f dB', psnr_b));
53 subplot(2,3,5); imshow(uint8(recon_o)); title(sprintf('Recon (Q=35)\n
    nPSNR: %.1f dB', psnr_o));
54 subplot(2,3,6); imshow(uint8(recon_p)); title(sprintf('Recon (Q=5)\n
    nPSNR: %.1f dB', psnr_p));
55
56 % quality_levels = [90 70 50 30 10];
57
58 % figure('Name', 'Sigiriya - quality levels');
59 % subplot(2, 3, 1);
60 % imshow(uint8(sigiriya_img));
61 % title('(a) Original image');
62
63 % fprintf('\nSigiriya at different quality levels:\n');
64 % for k = 1:numel(quality_levels)
65 %     q = quality_levels(k);
66 %     Qk = getQuantMatrix(Q50, q);
67 %     [rec_k, zeros_k, psnr_k] = evaluate_compression(sigiriya_img,
        Qk);
68
69 %     fprintf('    Quality = %3d: Zeros = %.2f%%, PSNR = %.2f dB\n', q
        , zeros_k, psnr_k);
70 %     subplot(2, 3, k + 1);
71 %     imshow(uint8(rec_k));
72 %     title(sprintf('(%c) Quality = %d, zeros: %.1f%%\nPSNR: %.1f dB
        ', ...
        'a' + k, q, zeros_k, psnr_k));
73 % end
74
75 %DEFINING functions
76
77
78 function img = crop8(img)
79     % Crop so both dimensions are multiples of 8
80     [r, c] = size(img);
81     img = img(1:floor(r/8)*8, 1:floor(c/8)*8);
82 end
83
84 function Q_new = getQuantMatrix(Q50, quality)
85     if quality > 50

```

```

86         tau = (100 - quality) / 50;
87     elseif quality < 50
88         tau = 50 / quality;
89     else
90         tau = 1;
91     end
92
93     Q_new = max(1, min(255, round(Q50 * tau)));
94 end
95
96 function [recon_img, percent_zeros, psnr_val] = evaluate_compression
    (orig_img, Q)
97     orig_img = double(orig_img);
98     [rows, cols] = size(orig_img);
99     recon_img = zeros(rows, cols);
100    total_elements = rows * cols;
101    zero_count = 0;
102
103    %STEP 1:
104    % Perform DCT and quantization on each 8x8 block
105    % Processing each 8x8 block
106    for r = 1:8:rows
107        for c = 1:8:cols
108            block = orig_img(r:r+7, c:c+7);
109
110            % Forward DCT and quantization
111            C = dct2(block - 128);
112
113            %STEP 2:
114
115            S = round(C ./ Q);
116
117            % Counting zeros in the quantized matrix
118            zero_count = zero_count + sum(S(:) == 0);
119
120            %STEP 4:
121            % Perform inverse quantization and inverse DCT
122            R = Q .* S;
123            E = idct2(R);
124
125            % Adding 128 to shift the pixel values back to the [0,
126            255] range
127            recon_img(r:r+7, c:c+7) = E + 128;
128        end
129    end
130
131    percent_zeros = (zero_count / total_elements) * 100;
132
133    mse = sum((orig_img(:) - recon_img(:)).^2) / total_elements;
134    if mse == 0

```

```
134         psnr_val = Inf;  
135     else  
136         psnr_val = 10 * log10((255^2) / mse);  
137     end  
138 end
```